```
!pip install -q transformers datasets evaluate accelerate
```

```
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 84.1/84.1 kB 3.9 MB/s eta 0:00:00
```

```
from transformers import pipeline
import torch
import pandas as pd # Already imported, but good to keep for context
```

```
try:
    import transformers
    import datasets
    import evaluate
    import accelerate
    print("Hugging Face Transformers and its dependencies are installed successfully!")
    print(f"Transformers version: {transformers.__version__}")
except ImportError:
    print("There was an issue importing Hugging Face Transformers or its dependencies.")
```

```
Hugging Face Transformers and its dependencies are installed successfully!
Transformers version: 5.0.0
```

```
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM

# Model identifier
model_name = "sshleifer/distilbart-cnn-12-6"

print("Loading tokenizer and model...")
# Load tokenizer
tokenizer = AutoTokenizer.from_pretrained(model_name)

# Load model
model = AutoModelForSeq2SeqLM.from_pretrained(model_name)
print("Tokenizer and model loaded.")

# Example text to demonstrate the summarizer
```

```
sample_text = """The history of artificial intelligence (AI) began in antiquity, with myths, stories, and rumors of artificial bei

print("\nGenerating summary for a sample text...")
# Tokenize the input text
inputs = tokenizer([sample_text], max_length=1024, return_tensors='pt', truncation=True)

# Generate the summary
summary_ids = model.generate(
    inputs['input_ids'],
    num_beams=4,
    max_length=130,
    min_length=30,
    early_stopping=True
)

# Decode the generated summary
summary_text = tokenizer.decode(summary_ids[0], skip_special_tokens=True)

print("\nOriginal Text (excerpt):")
print(sample_text[:500] + "...")
print("\nGenerated Summary:")
print(summary_text)
```

```
Loading tokenizer and model...
tokenizer_config.json: 100%  26.0/26.0 [00:00<00:00, 268B/s]
vocab.json: 899k/? [00:00<00:00, 28.8MB/s]
merges.txt: 456k/? [00:00<00:00, 17.9MB/s]
pytorch_model.bin: 100%  1.22G/1.22G [00:19<00:00, 64.4MB/s]
model.safetensors: 100%  1.22G/1.22G [00:18<00:00, 112MB/s]
Please make sure the generation config includes `forced_bos_token_id=0`.
Loading weights: 100%  358/358 [00:02<00:00, 151.73it/s, Materializing param=model.shared.weight]
The tied weights mapping and config for this model specifies to tie model.shared.weight to model.decoder.embed_tokens.weight, but
The tied weights mapping and config for this model specifies to tie model.shared.weight to model.encoder.embed_tokens.weight, but
Tokenizer and model loaded.

Generating summary for a sample text...

Original Text (excerpt):
The history of artificial intelligence (AI) began in antiquity, with myths, stories, and rumors of artificial beings endowed with

Generated Summary:
 The field of artificial intelligence (AI) was founded at Dartmouth College in 1956 . In the 1940s, scientists began to seriously
```

```
import pandas as pd

# Define example text passages as a list of dictionaries.
# Please expand this list with your own texts (5-8 passages recommended),
# ensuring they are approximately 150-400 words long and represent diverse sources.
# Each dictionary should have 'filename' (e.g., 'my_news_article.txt') and 'text' keys.
example_passages = [
    {
        'filename': 'passage_1.txt',
        'text': """The Amazon rainforest, a vast and unparalleled biome, plays a critical role in global climate regulation and
    },
    {
        'filename': 'passage_2.txt',
        'text': """Artificial intelligence (AI) is rapidly transforming various aspects of human society, from healthcare and fi
    },
```

```
    {
        'filename': 'passage_3.txt',
        'text': """The history of the Internet is a fascinating journey from a military research project to a global communicati
    }
]

# Create a pandas DataFrame from the example passages
passages_df = pd.DataFrame(example_passages)

print(f"Successfully created a DataFrame with {len(passages_df)} text passages.")
print("Displaying the first 5 rows of the DataFrame:")
display(passages_df.head())
```

```
Successfully created a DataFrame with 3 text passages.
Displaying the first 5 rows of the DataFrame:
```

	filename	text
0	passage_1.txt	The Amazon rainforest, a vast and unparalleled...
1	passage_2.txt	Artificial intelligence (AI) is rapidly transf...
2	passage_3.txt	The history of the Internet is a fascinating j...

Summarization Performance Analysis

Based on the generated summaries (short, medium, and long), evaluate the model's performance across different domains using the following criteria:

1. Performance by Domain

- **News Text:** Does the model capture the 'who, what, when, where, why'? Is the lead preserved?
- **Technical Text:** How does the model handle domain-specific jargon? Are technical definitions or processes simplified correctly or are they lost?
- **Narrative Text:** Does the summary maintain the story arc or character motivations? Is the 'vibe' of the story preserved?
- **Conversational Text:** Does the model accurately reflect the exchange of ideas between speakers, or does it struggle with the informal structure?

2. Key Observation Metrics

Metric	Observation Questions
Information Coverage	Does the summary touch upon all main points of the original text?
Coherence	Is the summary logically consistent and easy to read? Do the sentences flow well?
Loss of Key Details	Were any critical facts, names, or figures omitted that change the meaning?
Domain Handling	How well did the model translate specialized terminology into a summary format?

3. Impact of Parameters

- **Sampling (`do_sample=True`):** Did the medium summaries feel more 'natural' or 'creative' compared to the deterministic short/long summaries?
- **Length Constraints:** At what point (short/medium/long) did the model start to lose too much information or start repeating itself?

Experimenting with Summarization Parameters

Now, let's explore how different parameters influence the generated summaries. We'll focus on:

- `max_length`: The maximum length of the generated summary.
- `min_length`: The minimum length of the generated summary.
- `do_sample`: If `True`, use sampling; otherwise, use greedy decoding. Sampling introduces more randomness and creativity, potentially leading to more diverse summaries, while greedy decoding is deterministic and often produces more 'standard' summaries.

```
print("Generating summaries with different lengths and sampling parameters...")

# Define different parameter sets for experimentation
param_sets = {
    "short_summary": {"max_length": 50, "min_length": 10, "do_sample": False},
    "medium_summary": {"max_length": 100, "min_length": 30, "do_sample": True, "top_k": 50, "temperature": 0.7},
    "long_summary": {"max_length": 200, "min_length": 50, "do_sample": False}
}

# Prepare a list to store the experimental summaries
experimental_summaries_data = []
```

```
# Iterate through each passage and generate summaries with different parameters
for index, row in passages_df.iterrows():
    original_text = row['text']
    filename = row['filename']

    # Tokenize the input text once for efficiency
    inputs = tokenizer([original_text], max_length=1024, return_tensors='pt', truncation=True)

    passage_results = {
        'filename': filename,
        'original_text': original_text
    }

    for param_name, params in param_sets.items():
        print(f"Generating {param_name} for {filename}...")
        summary_ids = model.generate(
            inputs['input_ids'],
            num_beams=4 if not params.get('do_sample', False) else 1, # Use beam search for non-sampling, else greedy/sampling
            **params
        )
        summary_text = tokenizer.decode(summary_ids[0], skip_special_tokens=True)
        passage_results[param_name] = summary_text

    experimental_summaries_data.append(passage_results)

# Create a new DataFrame for experimental summaries
experimental_summaries_df = pd.DataFrame(experimental_summaries_data)

print("Experimental summaries generated successfully!")
print("Displaying the comparison of original texts and different summary lengths:")

# Display the results for comparison
for index, row in experimental_summaries_df.iterrows():
    print(f"\n--- Comparison for {row['filename']} ---")
    print("Original Text (excerpt):")
    print(row['original_text'][:500] + "...")
    print("\n--- Short Summary ---")
    print(row['short_summary'])
    print("\n--- Medium Summary (with sampling) ---")
```

```
    print(row['medium_summary'])
    print("\n--- Long Summary ---")
    print(row['long_summary'])
    print("--------------------------------------------------")
```

```
Generating summaries with different lengths and sampling parameters...
Generating short_summary for passage_1.txt...
Generating medium_summary for passage_1.txt...
Generating long_summary for passage_1.txt...
Generating short_summary for passage_2.txt...
Generating medium_summary for passage_2.txt...
Generating long_summary for passage_2.txt...
Generating short_summary for passage_3.txt...
Generating medium_summary for passage_3.txt...
Generating long_summary for passage_3.txt...
Experimental summaries generated successfully!
Displaying the comparison of original texts and different summary lengths:

--- Comparison for passage_1.txt ---
Original Text (excerpt):
The Amazon rainforest, a vast and unparalleled biome, plays a critical role in global climate regulation and biodiversity. Spanni

--- Short Summary ---
 The Amazon rainforest plays a critical role in global climate regulation and biodiversity . Spanning across nine South American

--- Medium Summary (with sampling) ---
 Amazon rain forests are home to an estimated 10% of the world's known species . The rainforest acts as a massive carbon sink, ab

--- Long Summary ---
 The Amazon rainforest plays a critical role in global climate regulation and biodiversity . Spanning across nine South American
--------------------------------------------------

--- Comparison for passage_2.txt ---
Original Text (excerpt):
Artificial intelligence (AI) is rapidly transforming various aspects of human society, from healthcare and finance to transportat

--- Short Summary ---
 AI offers immense potential for solving some of the world's most pressing challenges, such as disease diagnosis and climate mode

--- Medium Summary (with sampling) ---
 AI rapidly transforming various aspects of society, from healthcare to transportation and entertainment, is rapidly transforming

--- Long Summary ---
```

```
 AI offers immense potential for solving some of the world's most pressing challenges, such as disease diagnosis and climate mode
--------------------------------------------------

--- Comparison for passage_3.txt ---
Original Text (excerpt):
The history of the Internet is a fascinating journey from a military research project to a global communication network. It origi

--- Short Summary ---
 ARPANET was an initiative by the U.S. Department of Defense to create a robust, decentralized computer network capable of surviv

--- Medium Summary (with sampling) ---
 ARPANet, an initiative by the U.S. government, created a robust, decentralized computer network in 1960s . The Internet was crea

--- Long Summary ---
 ARPANET was an initiative by the U.S. Department of Defense to create a robust, decentralized computer network capable of surviv
--------------------------------------------------
```

```
print("Generating summaries for each passage in the DataFrame...")

# Prepare a list to store the summaries
summarized_data = []

# Iterate through each row in the DataFrame and generate a summary
for index, row in passages_df.iterrows():
    original_text = row['text']
    filename = row['filename']

    # Tokenize the input text
    inputs = tokenizer([original_text], max_length=1024, return_tensors='pt', truncation=True)

    # Generate the summary
    summary_ids = model.generate(
        inputs['input_ids'],
        num_beams=4,
        max_length=150, # Adjust max_length for generated summary as needed
        min_length=30,
        early_stopping=True
    )

    # Decode the generated summary
```

```
    summary_text = tokenizer.decode(summary_ids[0], skip_special_tokens=True)

    summarized_data.append({
        'filename': filename,
        'original_text': original_text,
        'generated_summary': summary_text
    })

# Create a new DataFrame for summaries
summaries_df = pd.DataFrame(summarized_data)

print("Summaries generated successfully!")
print("Displaying original texts and their generated summaries:")

# Display the results
for index, row in summaries_df.iterrows():
    print(f"\n--- {row['filename']} ---")
    print("Original Text (excerpt):")
    print(row['original_text'][:500] + "...")
    print("\nGenerated Summary:")
    print(row['generated_summary'])
    print("--------------------------------------------------")
```

```
Generating summaries for each passage in the DataFrame...
Summaries generated successfully!
Displaying original texts and their generated summaries:

--- passage_1.txt ---
Original Text (excerpt):
The Amazon rainforest, a vast and unparalleled biome, plays a critical role in global climate regulation and biodiversity. Spanni

Generated Summary:
 The Amazon rainforest plays a critical role in global climate regulation and biodiversity . Spanning across nine South American
--------------------------------------------------

--- passage_2.txt ---
Original Text (excerpt):
Artificial intelligence (AI) is rapidly transforming various aspects of human society, from healthcare and finance to transportat

Generated Summary:
 AI offers immense potential for solving some of the world's most pressing challenges, such as disease diagnosis and climate mode
--------------------------------------------------
```

```
--- passage_3.txt ---
Original Text (excerpt):
The history of the Internet is a fascinating journey from a military research project to a global communication network. It origi

Generated Summary:
 ARPANET was an initiative by the U.S. Department of Defense to create a robust, decentralized computer network capable of surviv
--------------------------------------------------
```

```
import pandas as pd

# A long passage (>400 words) about Space Exploration
long_passage_text = """The dawn of the space age was marked by the launch of Sputnik 1 by the Soviet Union in 1957, a feat that

# Create a new row for the long passage
new_row = pd.DataFrame([{
    'filename': 'long_passage_space.txt',
    'text': long_passage_text
}])

# Append to the existing passages_df
passages_df = pd.concat([passages_df, new_row], ignore_index=True)

print(f"Added long passage. Total passages in DataFrame: {len(passages_df)}")
display(passages_df.tail(1))
```

```
Added long passage. Total passages in DataFrame: 4
```

	filename	text
3	long_passage_space.txt	The dawn of the space age was marked by the la...

```
print("Summarizing the long passage...")

# Get the last added passage
long_text = passages_df.iloc[-1]['text']

# Tokenize
inputs = tokenizer([long_text], max_length=1024, return_tensors='pt', truncation=True)
```

```
# Generate summary (Long format)
summary_ids = model.generate(
    inputs['input_ids'],
    num_beams=4,
    max_length=200,
    min_length=50
)

summary_text = tokenizer.decode(summary_ids[0], skip_special_tokens=True)

print("\n--- Long Passage Summary ---")
print(summary_text)
```

```
Summarizing the long passage...

--- Long Passage Summary ---
 The dawn of the space age was marked by the launch of Sputnik 1 by the Soviet Union in 1957 . In 1961, Yuri Gagarin became the f
```